2026-03-08

INF-5183 – Fondements de l'Intelligence Artificielle

Projet de mi-session

INTITULE : ALGORITHMES DE RECHERCHE DANS UN LABYRINTHE

Réalisé par :

IVE MARCIAL NGOUAKANG

Enseignant:

PROF. MOHAMED LAMINE ALLAOUI

Abstract

Ce rapportporte sur la génération et la résolution d'un labyrinthe bidimensionnel dans un cadre d'intelligence artificielle et d'algorithmique. Les bordures extérieures sont fixées comme des murs, tandis que les murs internes sont générés de manière aléatoire afin de produire des configurations variées. Pour assurer la reproductibilité des résultats, le générateur prend en compte l'utilisation d'une graine aléatoire (seed). Une fois le labyrinthe généré, un algorithme de recherche est appliqué afin de déterminer un chemin entre S et G. Dans le cadre de ce projet, les méthodes de parcours de graphe, notamment BFS et DFS, permettent de modéliser efficacement l'exploration de la grille. Ce travail met ainsi en évidence l'importance de la représentation du problème, de la génération contrôlée d'un environnement valide et du choix d'un algorithme adapté pour la recherche de chemin. Les résultats obtenus montrent qu'il est possible de combiner génération aléatoire et résolution systématique dans un même programme, tout en respectant les contraintes imposées par l'énoncé. Le problème consiste à construire un labyrinthe de taille 16x16 comprenant des murs, des cases libres, un point de départ S et un point d'arrivée G, tout en garantissant l'existence d'un chemin valide entre ces deux positions. Une modélisation et implémentation sont conçues sur Python 3.11 simulé afin d'extraire les différentes analyses pour nous orienter sur les choix de l'algorithme optimal.

Introduction

Rechercher ou résoudre les problèmes dans un labyrinthe est une méthode courante en génie informatique et en intelligence artificielle. La résolution de ce labyrinthe met en exergue plusieurs notions telles que : la modélisation d'un environnement, la construction d'obstacles dans cet environnement, la recherche du chemin, la prise de décision dans cet espace au moyen de différents algorithmes de recherche. Dans la résolution de ce type de problème un agent se déplace d'un point de départ jusqu'à un point d'arrivée tout en respectant un certain nombre de principes, et contraintes telles que l'interdiction de ne pas traverser les murs, la limitation des déplacements par les obstacles. Un tel exercice est rencontré dans plusieurs domaines tels que les systèmes de navigation, robotique, jeux vidéo et bien plus d'autres en génie, etc.

L'intérêt de ce devoir est de générer un labyrinthe sous forme d'une matrice 16 x 16 où chaque case (coordonnée) constitue les bordures extérieures « murs » délimitant l'espace possible de recherche de l'agent. Les murs et les obstacles « infranchissables » sont représentés par « # ». Les cases libres « interne au labyrinthe » où l'agent peut se déplacer sont représentées par « . ». La case « coordonnée (1,1) » est la position initiale désignée par « S ». La case « coordonnée (14,14) » est la position finale désignée par « G ». L'agent devra trouver un chemin valide allant de S à G en se déplaçant uniquement dans les 4 directions cardinales (haut, bas, gauche, droite), sans traverser les murs.

L'objectif est de générer dans un premier temps la structure du labyrinthe en respectant les contraintes bien définies, ensuite faire la recherche ou trouver le chemin permettant de trouver une solution existante partant d'un point de départ S au point d'arrivée G satisfaisant les différents algorithmes de recherche.

METHODOLOGIE

L'élaboration de ce travail a consisté à poser différents scénarios possibles de résolution afin de trouver l'approche efficace pour mener à bien cet exercice.

Une première approche a été de comprendre les différentes contraintes et comment organiser le travail afin de les satisfaire. Initialement installer la version Python 3.11 comme requis par le professeur, ensuite écrire les instructions permettant la génération de la grille labyrinthe 16x16, ensuite le positionnement des obstacles internes de manière fixe, positionner les points de départ S et d'arrivée G, faire la recherche du chemin représenté par les (**.**)permettant de déplacer le point S vers G en navigant à travers des obstacles tout ceci dans un fichier **.py** et reproduire le même procédé cette fois en déployant les obstacle de manière aléatoire. La génération des murs intérieurs permet de produire une diversité de parcours du point initial au point final, seulement un problème majeur était celui d'avoir tous les points d'obstacle de manière à ne pas permettre le passage de l'agent. Ainsi pour garantir cela il a fallu fixer premièrement l'existence d'un chemin valide avant de peupler le reste du labyrinthe.

Une fois les premières simulations ont produit les résultats, j'ai procédé à l'exécution des contraintes concernant la répartition des différentes fonctions Python devant réaliser une tâche particulière tel que:

- La fonction de génération du labyrinthe : murs extérieurs, obstacles internes, point de départ, point d'arrivée et possible chemin. Elle est réalisée au travers de la fonction maze.py.
- La fonction de recherche algorithme BFS : qui rappelle la fonction maze.py, reproduit un premier labyrinthe pouvant reproduire toutes les différentes cases pouvant être explorées (**P**). Ensuite un deuxième labyrinthe pouvant reproduire le chemin solution (*****). Puis recalculer les coordonnées du chemin solution (**1,1**). Et finalement une structure qui regroupe le nombre de nœuds parcourus P, la longueur du chemin * et le temps mis pour exécuter la solution. Au travers de la fonction bfs.py.
- La fonction de recherche algorithme DFS : qui rappelle la fonction maze.py, reproduit un premier labyrinthe pouvant reproduire toutes les différentes cases pouvant être explorées (**P**). Ensuite un deuxième labyrinthe pouvant reproduire le chemin solution (*****). Puis recalculer les coordonnées du chemin solution (**1,1**). Et finalement une structure qui regroupe le nombre de nœuds parcourus P, la longueur du chemin * et le temps mis pour exécuter la solution. Au travers de la fonction dfs.py.
- La fonction de recherche algorithme A* (manhattan) : qui rappelle la fonction maze.py, reproduit un premier labyrinthe pouvant reproduire toutes les différentes cases pouvant être explorées (**P**). Ensuite un deuxième labyrinthe pouvant

reproduire le chemin solution (*****). Puis recalculer les coordonnées du chemin solution (**1,1**). Et finalement une structure qui regroupe le nombre de nœuds parcourus P, la longueur du chemin * et le temps mis pour exécuter la solution. Au travers de la fonction astar.py.

```
===== ALGORITHME DE RECHERCHE DU LABYRINTHE =====

          Labyrinthe généré :
          ==================

# # # # # # # # # # # # # # # #
# S . . . . . . # . . . # # # #
# . . . . # . # . # # # . . . #
# # . . . . . . # # . . . # . #
# # . . . . . . . . . . . . . #
# . # # # . # . . . # . . . . #
# # # . # . . . . . . # # . . #
# . # # . # # . . . . . # . . #
# # . # . . # . . . . . . # . #
# # # # . . . # # # # . # . . #
# . # # . # . . . . . . . . . #
# # . # . # . # . # . # . . . #
# . # . # . . # . # . . . . . #
# . # # . # . # # . # # . . . #
# . . . . # . . # # . . . . G #
# # # # # # # # # # # # # # # #
```

Maze.py a été réalisé en premier afin de comprendre le projet et la réalisation en respectant les contraintes. Tous les autres fichiers ou fonctions .py sont une variété de maze.py devant trouver la solution selon un algorithme particulier.

La contrainte finale a été la réalisation de la fonction **main. py**, qui rappelle toutes les fonctions : maze.py, bfs.py, dfs.py et astar.py et reproduit tous les résultats possibles calculés.

Une difficulté particulière a été la gestion et le positionnement des obstacles ou barrières de manière aléatoire (*seed*) à l'intérieur du labyrinthe, trouver les instructions pouvant réaliser les différents algorithmes. Il a été plus facile de comprendre cela au travers de différents forums de programmation auxquels je fais souvent et naturellement référence et les suggestions de chatgpt.

EXPLICATION DES ALGORITHMES

Les algorithmes ici explorés sont :

- **Algorithme BFS** (Breadth First Search) classé dans la catégorie des algorithmes de recherche non-informée effectue une recherche en explorant un parcours en largeur. Sa technique est d'explorer en premier toutes les cases situées à la même distance du point de départ en partant toujours de la gauche, ensuite explorer les pas -1, -2 en dessous ainsi de suite à chaque fois la solution n'est pas trouvée jusqu'à la

```
=== ============ ===
=== Algorithme BFS ===
=== ============ ===

=== Algorithme BFS EXPLORATION (p) ===
# # # # # # # # # # # # # # # #
# S p p p p # p # p p p # p p #
# p # p p p p p p p p p # p #
# p p p p p p p # p p p # p p #
# p p p p p p p # p p p p # #
# p p # # # p p p p p # p p p #
# p p p # p p p p p p p p p #
# # p # p p p p p p p p p p #
# p p p p # # p p p p p p p #
# p p p # # p p # p p # p p # #
# # p p p # p p p p p p p p #
# p p # p # p p # p # # p # #
# p p p p p p p p p # p # p #
# p # p p p p p # p p p p p #
# p p p # p p # p p p # # p G #
# # # # # # # # # # # # # # # #

=== Algorithme BFS SOLUTION (*) ===
# # # # # # # # # # # # # # # #
# S . . . . # . # . . . # . . #
# * # . . . . . . . . . # . #
# * . . . . . . # . . . # . . #
# * . . . . . . . # . . . . # #
# * . # # # . . . . . # . . . #
# * * . . # . . . . . . . . . #
# # * # . . . . . . . . . . # #
# . * . . . # # # . . . . . . . #
# . * . # # . . # . . # . . # #
# # * . . . # . . . . . . . . . #
# . * # . . . # . . . # . # # . #
# . * * * * * * * . # . # . #
# . # . . . . . # * * * * * . #
# . . . # . . # . . . . # # * G #
# # # # # # # # # # # # # # # #

=== Algorithme BFS Coordonnées CHEMIN ===
Chemin : S(1, 1) -> (2, 1) -> (3, 1) -> (4, 1) -> (5, 1) -> (6, 1) -> (6, 2) -> (7, 2) -> (8, 2) -> (9, 2) -> (10, 2) -> (11, 2) -> (12, 2) -> (12, 3) -> (12, 4) ->
 (12, 5) -> (12, 6) -> (12, 7) -> (12, 8) -> (12, 9) -> (13, 9) -> (13, 10) -> (13, 11) -> (13, 12) -> (13, 13) -> (14, 13) -> G(14, 14)
```

trouver. Cette méthode garantit de trouver le chemin le plus court en termes de nombre de déplacements (cases), c'est une solution simple efficace et fiable.

- **L'algorithme DFS** (Depth First Search) classé dans la catégorie des algorithmes de recherche non-informée effectue une recherche en explorant un parcours en profondeur. Sa technique consiste à explorer un chemin solution le plus loin possible en profondeur sur la branche avant de revenir en arrière au cas où la solution n'est pas trouvée. Il ne garantit pas toujours le chemin le plus court.

```
                    === ============= ===
                    === Algorithme DFS ===
                    === ============= ===

=== Algorithme DFS EXPLORATION (p) === ===
# # # # # # # # # # # # # # # #
# S p p p p # . # . . . # . . #
# . # . . p p p p p p p # . #
# . . . . . . # . . p # . . #
# . . . . . . # . . p p p # #
# . . # # # . . . . . # . p p #
# . . . # . . . . . . . . . p #
# # . # p p p p p p p p p p #
# p p p p # # # . . . . . . . #
# p p . # # . . # . . # . . # #
# # p p p # . . . . . . . . . #
# . . # p p # . . . # . # # . #
# . . . p p p p p p # . # . #
# . # . . . . # . p p p p p #
# . . . # . . # . . . # # . G #
# # # # # # # # # # # # # # # #

=== Algorithme DFS SOLUTION (*) ===
# # # # # # # # # # # # # # # #
# S * * * * # . # . . . # . . #
# . # . . * * * * * * * . # . #
# . . . . . . . # . . . # . . #
# . . . . . . . # . . * * * # #
# . . # # # . . . . . # . * * #
# . . . # . . . . . . . . . . *
# # . # * * * * * * * * * * * #
# . * * * # # # . . . . . . . #
# . * . # # . . # . . # . . # #
# # * * * # . . . . . . . . . #
# . . # * * # . . . # . # # . #
# . . . . * * * * * * # . # . #
# . # . . . . . # . * * * * * #
# . . . # . . # . . . # # . G #
# # # # # # # # # # # # # # # #

=== Algorithme DFS Coordonnées CHEMIN ===
Chemin : S(1, 1) -> (1, 2) -> (1, 3) -> (1, 4) -> (1, 5) -> (2, 5) -> (2, 6) -> (2, 7) -> (2, 8) -> (2, 9) -> (2, 10) -> (2, 11) -> (3, 11) -> (4, 11) -> (4, 12) ->
 (4, 13) -> (5, 13) -> (5, 14) -> (6, 14) -> (7, 14) -> (7, 13) -> (7, 12) -> (7, 11) -> (7, 10) -> (7, 9) -> (7, 8) -> (7, 7) -> (7, 6) -> (7, 5) -> (7, 4) -> (8,
 4) -> (8, 3) -> (8, 2) -> (9, 2) -> (10, 2) -> (10, 3) -> (10, 4) -> (11, 4) -> (11, 5) -> (12, 5) -> (12, 6) -> (12, 7) -> (12, 8) -> (12, 9) -> (12, 10) -> (13, 1
0) -> (13, 11) -> (13, 12) -> (13, 13) -> (13, 14) -> G(14, 14)
```

- **L'algorithme A* Manhattan** classé dans la catégorie des algorithmes de recherche informée effectue une recherche de coût **f(n)** = g(n) + h(n) où n est la case, f(n) est le coût total estimé, h(n) est l'estimation du coût restant jusqu'à l'objectif et g(n) est le coût réel depuis le départ jusqu'à la case n. Il combine les cases qui ont déjà été visitées et celles à probablement visiter. Il calcule les coûts et choisit en priorité celle qui apparait avec le meilleur score.

```
                    === ======================= ===
                    === Algorithme A* (Manhattan) ===
                    === ======================= ===

=== Algorithme A* (Manhattan) EXPLORATION (p) === ===
# # # # # # # # # # # # # # # #
# S p p p p # . # . . . # . . #
# p # p p p p p p p p p # . #
# p p p p p p p # p p p # . . #
# p p p p p p p # p p p p p # #
# p p # # # p p p p p # p p p #
# p p p # . p p p p p p p p p #
# # p # . . p p p p p p p p p #
# . p p p # # # p p p p p p p #
# . p p # # . . # p p # p p # #
# # p p p # . . . p p p p p p #
# . p # p p # . . . p # p # # p #
# . p p p p p p p p # . # p #
# . # p p p p p # p p p p p p #
# . . p # p p # . p p # # p G #
# # # # # # # # # # # # # # # #

=== Algorithme A* (Manhattan) SOLUTION (*) ===
# # # # # # # # # # # # # # # #
# S * * * * # . # . . . # . . #
# . # . . * * * * * * * . # . #
# . . . . . . . # . . * # . . #
# . . . . . . . # . . * * * # #
# . . # # # . . . . . # . * . #
# . . . # . . . . . . . . * . #
# # . # . . . . . . . . . * . #
# . . . . # # # . . . . . * . #
# . . . # # . . # . . # . * # #
# # . . . # . . . . . . . * * #
# . . # . . # . . . # . # # * #
# . . . . . . . . . . # . # * #
# . # . . . . . # . . . . . * #
# . . . # . . # . . . # # . G #
# # # # # # # # # # # # # # # #

=== Algorithme A* (Manhattan) Coordonnées CHEMIN ===
Chemin : S(1, 1) -> (1, 2) -> (1, 3) -> (1, 4) -> (1, 5) -> (2, 5) -> (2, 6) -> (2, 7) -> (2, 8) -> (2, 9) -> (2, 10) -> (2, 11) -> (3, 11) -> (4, 11) -> (4, 12) ->
 (4, 13) -> (5, 13) -> (6, 13) -> (7, 13) -> (8, 13) -> (9, 13) -> (10, 13) -> (10, 14) -> (11, 14) -> (12, 14) -> (13, 14) -> G(14, 14)
```

TABLEAU COMPARATIF DES ALGORITHMES

À la suite des différents algorithmes construits a été évalué le nombre de nœuds explorés P, le nombre de cases constituant le chemin solution * et le temps d'exécution de chacun de ces algorithmes et reporté dans le tableau ci-dessous.

```
                                  ===== ================== =====
                                  ===== TABLEAU COMPARATIF =====
                                  ===== ================== =====
ALGORITHME                        NOEUDS                  LONGUEUR                TEMPS(ms)
DFS                                47                      42                      0.243 ms
BFS                                133                     26                      5.615 ms
A* (Manhattan)                     68                      26                      0.411 ms
```

PARTIE OPTIONNELLE (pas demander dans le devoir)

1- Afin de rendre plus lisibles les résultats ou particulièrement les chemins de solution, j'ai opté pour une couleur verte pour mettre en évidence le chemin parcouru.

2- Un calcul supplémentaire permettant de mettre en évidence le meilleur temps (couleur verte) optimal des 3 algorithmes ainsi que le pire (couleur rouge). Voir le tableau ci-dessous.

```
                                  OPTIMISATION COUTS
TEMPS OPTIMAL                                                    0.243
TEMPS NON OPTIMAL                                                5.615
```

ANALYSES DES RESULTATS

Au regard des performances des 3 algorithmes de recherche ici utilisés, on remarque :

- Le nombre de nœuds explorés est différent pour chacun des 3 :
  - BFS a au total 133
  - DFS a au total 47
  - A* Manathan a au total 68

  Sur ce point l'algorithme DFS explore moins de nœuds que les 2 autres.

- Le nombre de nœuds chemin solution optimisé est différent pour chacun des 3 :
  - BFS a au total 26
  - DFS a au total 42
  - A* Manathan a au total 26

  Sur ce point l'algorithme BFS et A* Manhattan ont le moins de nœuds solution que DFS.

- Le temps d'exécution est différent pour chacun des 3 :

- BFS est de 5,615ms
- DFS est de 0.243ms
- A* Manathan est de 0.411ms

Sur ce point l'algorithme DFS a le temps optimal par rapport aux 2 autres.

Si la solution recherchée est basée sur le temps d'exécution force est de constater que l'algorithme DFS est optimal comparé aux 2 autres, mais si c'est plutôt la longueur du chemin on prendra plutôt A*.

CONCLUSION

L'élaboration de ce projet est un condensé à la fois de la partie théorique (cours) combinée à celle pratique (python) afin de comprendre le mécanisme des algorithmes de recherche longtemps utilisés dans le génie spécialement en robotique, planification automatique, l'intelligence artificielle. D'une part il est important de comprendre parfaitement le sujet ou la problématique avant de poser une possible solution, cela permet de simplifier énormément le projet et surtout de savoir où regarder en cas de révision.

Étant donné que la génération des obstacles était faite de manière aléatoire à chaque simulation, ceci nous permettait d'avoir également différents résultats à chaque fois.

Au regard des résultats observés plusieurs fois, il a été constaté que le nombre de nœuds n'est pas lié proportionnellement au chemin solution et encore moins au temps d'exécution. Cela dépend fortement du type d'algorithme de recherche, de la position du point de départ et d'arrivée sans ignorer également la position des obstacles.

À chaque simulation les résultats du tableau ci-dessous changent en fonction de la disposition des obstacles et ceci définit l'algorithme optimal.

```
                                  ===== ================== =====
                                  ===== TABLEAU COMPARATIF =====
                                  ===== ================== =====
ALGORITHME                        NOEUDS                LONGUEUR              TEMPS(ms)
DFS                                47                    42                    0.243 ms
BFS                                133                   26                    5.615 ms
A* (Manhattan)                     68                    26                    0.411 ms
```

LIEN GITHUB

Ici vous trouvez le lien Github contenant le dossier Devoir_I où se trouvent tout le code source et d'autres fichiers supplémentaires.

https://github.com/MaxMaxCoding/Devoir_I.git